CSE 8843
Assignment #3

Group # 5

Aldo Adriazola

Tess Harris

Michael Hedrick

Dean Patterson

Problem 1

(2 Point) The overhead function of an isoefficient algorithm is:

$$T_o = p^{\frac{3}{2}} + p + p^{\frac{3}{4}} W^{\frac{3}{4}} + p^{\frac{3}{5}} + p^{\frac{1}{2}} W^{\frac{1}{2}}$$

Determine the asymptotic isoefficiency function of this parallel algorithm.

$W = K \cdot T_0(W, p)$

where W is the workload, p is the number of processors, and K is a constant related to the target efficiency.

First, we examine the terms that contain only p.

- $p^{\frac{3}{2}}: W \propto p^{\frac{3}{2}}$ $\Rightarrow \Theta(p^{1.5})$
- $p: W \propto p$ $\Rightarrow \Theta(p)$
- $p^{\frac{3}{5}}: W \propto p^{\frac{3}{5}}$ $\Rightarrow \Theta(p^{0.6})$

For those terms, $p^{\frac{3}{2}}$ will grow the fastest. Next, examine the terms with both W and p.

- $p^{\frac{3}{4}} W^{\frac{3}{4}}$:
  - $W = K \cdot p^{\frac{3}{4}} W^{\frac{3}{4}}$
  - $W / W^{\frac{3}{4}} = K \cdot p^{\frac{3}{4}}$
  - $W^{\frac{1}{4}} = K \cdot p^{\frac{3}{4}}$
  - $\left(W^{\frac{1}{4}}\right)^4 = \left(K \cdot p^{\frac{3}{4}}\right)^4$
  - $W = K^4 \cdot p^3$ $\Rightarrow \Theta(p^3)$
- $p^{\frac{1}{2}} W^{\frac{1}{2}}$:
  - $W = K \cdot p^{\frac{1}{2}} W^{\frac{1}{2}}$
  - $W / W^{\frac{1}{2}} = K \cdot p^{\frac{1}{2}}$
  - $W^{\frac{1}{2}} = K \cdot p^{\frac{1}{2}}$
  - $\left(W^{\frac{1}{2}}\right)^2 = \left(K \cdot p^{\frac{1}{2}}\right)^2$
  - $W = K^2 \cdot p$ $\Rightarrow \Theta(p)$

The $p^{\frac{3}{4}} W^{\frac{3}{4}}$term cause the workload to grow at a rate of p^3, faster than any other term in the overhead function. The asymptotic isoefficiency function is: $\Theta(p^3)$.

Problem 2

(6 Points) Two sequential algorithms A_1 and A_2 for solving the same problem yield the following expressions for their runtimes:

$$A_1 : T_{s1} = an^2 + bn + c$$

$$A_2 : T_{s2} = a'n^2 \log(n) + b'n^2 + c'n + d'$$

$$where \begin{cases} a, b, c \\ a', b', c', d' \end{cases} \text{ are constants}$$

A parallel algorithm for the same problem yields the following expression for the runtime:

$$T_p = \frac{n^2}{p} + 64 \log(p)$$

(a) Is this parallel algorithm cost-optimal with respect to A_1 and/or A_2? For each case, if the answer is yes, give the constraints for which your algorithm is cost-optimal.

(b) Is this algorithm cost-optimal with respect to A1 and A2 for values of n for which the parallel running time is minimum? Explain.

(c) Discuss the relevance of the above analysis given that, in general, only the best sequential algorithm is considered for the scalability analysis of a parallel algorithm.

Answer:

(a) Determine the cost for the parallel algorithm.

$$C_p = p\left(\frac{n^2}{p} + 64 \log p\right) = n^2 + 64\, p \log p$$

Compared to A_1. For A_1, the n^2 term dominates, so $T_{s1} = \Theta(n^2)$. For the parallel algorithm to be cost-optimal with respect to A_1, the parallel algorithm must also be $\Theta(n^2)$. This will be true as long as the overhead term $64\, p \log p$ does not asymptotically exceed the n^2 term. $n^2 = \Omega(p \log p)$

Compared to A_2. For A_2, the dominant term is $n^2 \log n$. For the parallel algorithm to be cost-optimal with respect to A_1, the parallel algorithm must be $\Theta(n^2 \log n)$. But the parallel algorithm is $\Theta(n^2)$, so it is not cost optimal with respect to A_2.

(b) We need to find the minimum run time by determining the optimal number of processors. We take the derivative of $T_p = \frac{n^2}{p} + 64 \log p$ and setting it equal to zero.

$$\frac{dT_p}{dp} = -\frac{n^2}{p^2} + \frac{64}{p} = 0$$

$$\frac{64}{p} = \frac{n^2}{p^2} \Rightarrow \boldsymbol{p = \frac{n^2}{64}}$$

Now we calculate the parallel cost at the minimum by substituting $\frac{n^2}{64}$ for p in $n^2 + 64\, p \log p$.

$$C_p = n^2 + 64\left(\frac{n^2}{64}\right)\log\left(\frac{n^2}{64}\right)$$
$$C_p = n^2 + n^2 \log\left(\frac{n^2}{64}\right)$$
$$C_p = n^2 + n^2(\log n^2 - \log 64)$$
$$C_p = n^2 + 2n^2 \log n - n^2 \log 64$$
$$C_p = \Theta(n^2 \log n)$$

Compared to A_1. A_1 is $\Theta(n^2)$. The parallel algorithm in the $\Theta(n^2 \log n)$ when the parallel running time is minimum. This is not cost optimal, because the minimum running time of the parallel algorithm is asymptotically higher than the sequential runtime.

Compared to A_2. A_{12} is $\Theta(n^2 \log n)$. This matches the minimum running time of the parallel algorithm, so in this case, it is cost optimal. This holds where $p = \Theta(n^2)$. If p grows at a slower rate, then the n^2 term will dominate

(c) The table compares cost optimality of the parallel algorithm to A_1 and A_2 in the general case and at minimum time for the parallel algorithm.

Scenario	Compared to A_1 $\Theta(n^2)$.	Compared to A_2 $\Theta(n^2 \log n)$
General Case	Yes, if $n^2 \geq \Omega(p \log p)$	No
At Min T_p	No	Yes

In parallel computing, we only use the best sequential algorithm A_1 as a baseline to ensure we are not comparing against a suboptimal, slower one A_2. At the point of minimum runtime, the parallel algorithm becomes so burdened by overhead that its cost matches the inefficient A_2 instead of the optimal A_1. This highlights a fundamental trade-off – an algorithm can be optimized for absolute speed, but doing so can violate the efficiency required for scalability.

Problem 3

(4 Points) Matrix multiplication on the *PRAM*:

(a) Consider the matrix multiplication on the *PRAM* model, with *A* and *B* as two *n*×*n* matrices and *C* = *AB*. Show and explain how this computation can be executed in *O*(log(*n*)) time.

(b) By analyzing this parallel algorithm in terms of time and resources it takes, can you state whether it is efficient, and can you identify to which class of algorithms it belongs?

Answer:

(a) The goal is to calculate each element of the resulting matrix C, where $C_{i,j} = \sum_{k=1}^{n}\left(A_{i,j} \times B_{j,k}\right)$. First, we look at the total number of multiplications needed. For an $n \times n$ matrix, there are n^2 cells, and each cell needs n multiplications. That is a total of n^3 independent multiplication operations. Each processor can be assigned to each (i, j, k) triplet. All multiplications can happen concurrently, resulting in constant time $O(1)$. Now, the results have to be summed. If the summing were done sequentially, it would take $O(n)$ time. We could instead use a binary tree. First, we pair up the input to be summed. Then, we use n/2 processors to sum those pairs. The next step would use n/4 processors to sum up the results from the first level. We continue until all sums are performed. This would be the height of a binary tree which is $O(\log(n))$. So overall, we have:

$$\boldsymbol{O(\log(n)) + O(1) \Rightarrow O(\log(n))}.$$

(b) The algorithm performs n^3 operations, which align withs the definition of $W(n) = \Theta(T^*(n))$ and therefore is efficient in the weak sense. It achieves a running time of $O(\log(n))$. Based on the information contained in the Models 5 handout, the algorithm is efficient in the strong sense as well since it requires $\Omega(\log(n))$ to compute the sum regardless of the number of processors. Class NC is the class of problems solvable by a parallel Based on the definition stated in the NP Completeness handout, the algorithm is class NC: *Class NC is the class of problems solvable by a parallel algorithm on a PRAM using a polynomial number of processors in polylogarithmic worst-case time complexity processors in polylogarithmic worst-case time complexity.* In this case, the polynomial number of processors is n^3 and the polylogarithmic time is $O(\log(n))$, where the the exponent for the logarithm is 1.

Problem 4

(4 Points) The sequential complexity of a problem is $O\left(n^2\right)$. The problem can be solved by two parallel algorithms: P_{a1} and P_{a2}. These algorithms are characterized, respectively, by the following number of operations (work) and execution time.

$$P_{a_1} : \begin{cases} W_1(n) = an^2 + bn + c \\ T_{P_1}(n) = O\left(n^2\right) \end{cases}$$

$$P_{a_2} : \begin{cases} W_2(n) = mn^2 + pn + q \\ T_{P_2}(n) = O\left(n \log(n)\right) \end{cases}$$

$$where \begin{cases} a, b, c \\ m, p, q \end{cases} \text{ are constants}$$

For each of these parallel algorithms, P_{a1} and P_{a2}, determine whether they are:

- *optimal* in the *weak sense*
- *optimal* in the *strong sense*

Justify your answers.

Answer:

Definitions from Models 5

- **Optimal in the Weak Sense (WT-Optimal):** An algorithm is optimal in the weak sense if the total number of operations (work) $W(n) = \Theta(T^*(n))$. The total number of operations used by the optimal parallel algorithm is asymptotically the same as the sequential complexity of the problem, regardless of the running time $T(n)$ of the parallel algorithm.
- **Optimal in the Strong Sense:** A parallel algorithm is optimal in the strong sense if it is work-optimal (weak sense) and $T(n) = O(log\, n)$.

Analysis of P_{a1}

- Work: $W_1(n) = an^2 + bn + c \quad \Rightarrow \Theta(n^2)$
- Execution Time: $T_{P1}(n) = O(n^2)$
- It is optimal in the weak sense because the work $W_1(n) = \Theta(n^2)$ is asymptotically the same as the sequential complexity of the problem $O(n^2)$ as stated in the problem definition.
- It is not optimal in the strong sense because the processing time is $O(n^2)$ and is not $O(log\, n)$.

Analysis of P_{a2}

- Work: $W_2(n) = mn^2 + pn + q \quad \Rightarrow \Theta(n^2)$
- Execution Time: $T_{P2}(n) = O(log\, n)$
- It is optimal in the weak sense because the work $W_2(n) = \Theta(n^2)$ is asymptotically the same as the sequential complexity of the problem $O(n^2)$ as stated in the problem definition. (same as P_{a1})
- It is optimal in the strong sense because the processing time is $O(log\, n)$.

Problem 5

(4 Points) A scientific problem can be solved in two dimensions (2D) or three dimensions (3D), on a full quad-tree and full oct-tree, respectively. The solution to the problem requires the traversal of a full balanced tree and can make use of a parallel algorithm. Discuss the amount of concurrency available in the problem when using the (2D) version of the problem versus its 3D version, considering that both full balanced trees (2D and 3D) have the same number of leaves. Discuss also the implication that the concurrency of this problem has on the running time of the parallel algorithm.

Answer:

Let's consider an example of $N = 2^{12}$.
1D is a binary tree. The base b is 2. Using this is a baseline reference.
2Dis a quad-tree. The base b is 4.
3D is an oct-tree. The base b is 8.

For all types of trees above, the number of leaves is the same. To determine the height of the of a tree h is determined by taking the logarithm with respect to the base. $h = \log_b N$

Let's consider an example of $N = 2^{12}$.

For a binary tree: $h = \log_2(2^{12}) = 12$
For a quad-tree: $h = \frac{\log_2(2^{12})}{\log_2(4)} = \frac{12}{2} = 6$
For an oct-tree: $h = \frac{\log_2(2^{12})}{\log_2(8)} = \frac{12}{3} = 4$

Comparing 2D and 3D.

In a parallel environment with P processors, the oct-tree will saturate the available hardware much faster (in fewer steps) than the quad-tree.

- Quad-tree (2D): At the level $h = 4$, there are $4^4 = 256$ parallel tasks.
- Oct-tree (3D): At the level $h = 4$, there are $8^4 = 4096$ parallel tasks. This is the maximum concurrency.

Impact on Running Time

Assuming a parallel traversal where each level represents a synchronization point:

- The running time T is proportional to the height h. Since the height of the quad-tree is 6 and the height of the oct-tree, the oct-tree reduces the execution path by 33% compared to the quad-tree.
- Every level transition in a parallel algorithm typically incurs a synchronization cost. The 2D version requires 6 such barriers, whereas the 3D version requires only 4.
- If the system has P processors, the oct-tree allows you to achieve the result in 4 steps (plus overhead). The quad-tree would take 6 steps, making the 3D algorithm fundamentally faster for the same problem size N.